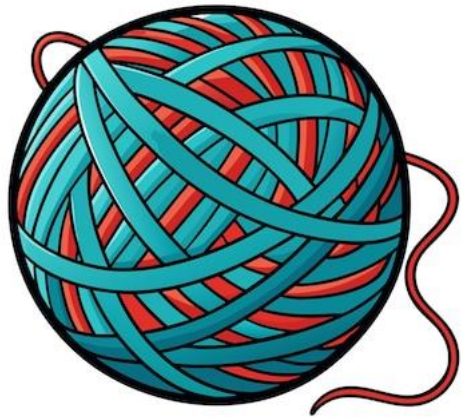
04. strings

```
Data Sciences Institute
Python
```

Learning objectives

- Understand what a string is and create single-line and multi-line strings
- Use operators, escape sequences, and string methods to modify strings
- Pass strings to functions and return or process them
- Learn how to format strings

What is a string?



- **String (str)** = text in Python
- A sequence of **letters, numbers, symbols, or spaces**
- Surrounded by **single** (') or **double** (") quotes
- Quotes must **match** and be **consistent**
- Switch quotes if text contains apostrophes or quotes

```
"this is a string"
```

```
'so is this'
```

```
"these quotes do not match' #SyntaxError
```

```
"Let's use double quotes to avoid error"
```

Multi-line strings

- Strings can **span multiple lines** using **triple quotes**:
  - ''' or """
- Useful for **long text** or **docstrings**

```
"""
This is a
multiline string
in Python.
"""
```

Example

```
my_story = """Once upon a time,
in a land far away,
there lived a curious cat."""
```

```
my_story
#Output is 'Once upon a time,\nin a land far
away,\nthere lived a curious cat.'
```

Where did those `'\n'` characters come from and where did our line breaks go in that last string?

Escape sequences

- **An escape sequence** is a special combination of characters that represents something else.
- The backslash \ is the escape character.
- **\n means new line**

```
print("Hello\nWorld")

#Output is
Hello
World
```

Escape sequences

Escape sequence	Description	Example	Output
\’	Single quote	`'It\'s Python!'`	`It's Python!`
\”	Double quote	`"He said \"Hi!\""`	`He said "Hi!"`
\\	Backslash	`"This is a backslash: \\"`	`This is a backslash: \`
\t	Tab	`"Name:\tAlice"`	`Name:    Alice`
\n	New line	`"Hello\nWorld"`	`Hello` `World`

Working with Strings & Operator Overloading

- Some **arithmetic and comparison operators** also work with strings.
- When an operator **does different things depending on the data type**, we say it is *overloaded*.
- Arithmetic operators:

```
# Concatenation (+) with strings
"Hello" + " " + "World"    # Output: "Hello World"
```

```
# Repetition (*) with strings
"ha" * 3                   # Output: "hahaha"
```

```
# Mixing data types results in an error
"The year is" + 2026       # Output TypeError
```

Working with Strings & Operator Overloading

- Some **arithmetic and comparison operators** also work with strings.
- When an operator **does different things depending on the data type**, we say it is *overloaded*.
- Comparison operators:

```
# Strings can be compared as being equal
"apple" == "Apple"    # Output: False
```

```
# Strings can be compared as being unequal
"apple" != "Apple"    # Output: True
```

```
# Tip: Always check the data type when comparing!
"20" == 20            # Output: False
```

Strings within strings (Slicing)

- We can **extract parts of a string using slicing**.
- Square brackets `[ ]` are used with indexes to get characters.
- Single character: use the index of the character.
- Multiple characters: use start:end inside the brackets.

```
name = "Alice"
```

```
A   l   i   c   e
```




```
name[0] # Output: A
```

```
name[0:3] # Output: Ali
```

Note. Slices do not include the character at the ending index position.

Strings within strings (Slicing)

- Python lets us also count from the end using **negative indices.**
- -1 is the last character, -2 is the second-to-last, and so on.
- Negative indices are very useful for slicing from the end without knowing the string length.

```
name = "Alice"
```

```
A    l    i    c    e
```

-5 -4 -3 -2 -1

```
name[-1] # Output: last character (e)
```

```
name[-3:]# Output: last 3 characters (ice)
```

Note. If you **omit the end** in a slice (start:), Python goes **all the way to the last character**.

Exercise

- Try this on your own:
    - Get the first 6 letters of this word: "supercalifragilisticexpialidocious"
    - Get the area code for this phone number (647)-225-7154

Checking for strings (substrings)

- You can check if a **substring** exists in a string using the **in operator**.
- Returns **True** if found, **False** if not.

```
job_qualifications = 'The successful applicant
will be proficient in R, Python, SQL, statistics,
and data visualization.'
```

```
'R' in job_qualifications              # Output: True
```

```
'r' in job_qualifications              # Output: True
```

```
'JavaScript' in job_qualifications # Output: False
```

Strings and functions

- Strings can be used with functions we've seen before, like `print()`.
- Strings also work with built-in functions, such as `len()`, which returns the **number of characters in a string**.

```
len('Sometimes we type long-winded sentences to
meet word counts or character counts.')
# Output: 80
```

```
len(name)# Recall name = 'Alice'
# Output: 5
```

Strings methods

- **Methods are functions that work only on a specific type of data.**
- **String methods work only on strings** (not on numbers or booleans).
- Methods are called using dot notation: `string.method()`

```
# Convert to uppercase
'I am not yelling'.upper()# Output: I AM NOT YELLING
```

```
# Count the 'e's
'This string is unusual'.count('e')# Output: 0
```

```
# Check if a string ends with a substring
'file_name.csv'.endswith('.csv')# Output: True
```

```
# Replace text
'file name'.replace(' ', '-')# Output: file-name
```

Strings methods

- Full list of string methods here: https://docs.python.org/3/library/stdtypes.html#string-methods
- Use Python's **help system** to learn about string methods

```
# Show all string methods
help(str)
```

```
# Show details about a specific string method
help(str.upper)
```

Formatting strings

- String formatting lets us **combine text with other data**.
- Two common ways, achieve the exact same outcome

1. **.format() method**

   uses `{}` s placeholders, then passes values to `.format()`

```
first_name = "Ada"
last_name = "Lovelace"

"Initials are {}.{}".format(first_name[0], last_name[0])
# Output 'Initials are A.L'
```

Formatting strings

- String formatting lets us **combine text with other data**.
- Two common ways, achieve the exact same outcome

2. **f-strings** *Note*. This requires Python 3.6+

   Concise and readable.

   Pass variables directly with `{}` inside an `f'...'` string

```
first_name = "Ada"
last_name = "Lovelace"

f"Initials are {first_name[0]}.{last_name[0]}"
# Output 'Initials are A.L'
```